

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

图的广度优先搜索

Breadth-First Search: Shortest Path in Unweighted Graphs

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. BFS 基本思想
2. BFS 的实现
3. 网格上的 BFS
4. 经典例题
5. 总结



目录

1 BFS 基本思想
从 DFS 到 BFS
队列与逐层扩展
2 BFS 的实现
3 网格上的 BFS
4 经典例题
5 总结

回顾：DFS 留下了什么

上一讲我们已经掌握：

- 图的存储：邻接表、邻接矩阵
- DFS：栈 / 递归，一条路走到底再回溯
- 访问标记 `vis[]`：避免重复访问与死循环

DFS 能“走通”，但不擅长“走最短”。

DFS 与 BFS 的对比

核心差异

- DFS 用**栈**：后进先出
- BFS 用**队列**：先进先出
- DFS 深优先，BFS 层优先

维度	DFS	BFS
结构	栈 / 递归	队列
顺序	深优先	层优先
最短路	不保证	**保证**
复杂度	$O(V+E)$	$O(V+E)$

BFS 的核心机制：队列

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

三步走

1. 起点入队，标记已访问
2. 取出队首 u，遍历其邻居
3. 未访问的邻居：标记 + 入队

重复第 2 步，直到队列为空。

队列的 FIFO 保证

- 先入队的点先扩展
- 距离为 k 的点全部先于距离 $k+1$ 的点
- 于是访问顺序天然分层

为什么 BFS 能求无权图最短路？

关键不变量：点 u 第一次被访问时，$dist[u]$ 就是 $s \to u$ 的最短距离。

直觉解释

- 队列保证了“按距离从小到大”出队
- 若 $s \to u$ 存在更短路径，该路径上的点会更早入队
- 那么 u 也会更早被发现，与假设矛盾

前提：所有边权相等（通常为 1）。

目录

1 BFS 基本思想

2 BFS 的实现
邻接表上的 BFS

3 网格上的 BFS

4 经典例题

5 总结

实现要素

必备三件套

- `queue<int> q`：待扩展的点
- `vis[]` 或 `dist[]`：是否访问
- `adj[]`：邻接表

最关键的一条

- 入队时就打标记
- 不要等出队才标记
- 否则同一点会被多次入队

用 `dist[]` 同时承担“访问标记 + 距离记录”两个职责，省一个数组。

代码：邻接表 BFS

```cpp
// adj 为邻接表，n 为点数，s 为起点
vector<vector<int>> adj;
vector<int> dist;

void bfs(int s, int n) {
    dist.assign(n, -1);       // -1 表示尚未访问
    queue<int> q;
    dist[s] = 0;
    q.push(s);                // 入队即标记

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] != -1) continue;   // 已访问，跳过
            dist[v] = dist[u] + 1;
            q.push(v);
```

代码：邻接表 BFS


```
19              }
20          }
21      }
```

三个常见错误

错误 1：出队才标记

- 同一点被多次入队
- 队列规模爆炸
- 复杂度从 $O(V+E)$ 退化

错误 2：忘记初始化

- `dist` 残留上一轮的值

错误 3：无向图只加单向边

- `adj[u].push_back(v)` 之后
- 忘记 `adj[v].push_back(u)`

入队即标记，最容易忘。

目录

1 BFS 基本思想

2 BFS 的实现

3 网格上的 BFS
- 网格建模
- 网格 BFS 实战

4 经典例题

5 总结

网格 = 隐式图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

对应关系

- 每个格子 → 一个点
- 上下左右相邻 → 一条边
- 每步代价为 1

为什么叫“隐式”

- 不需要真的建邻接表
- 用方向数组现算邻居
- 省内存，代码更短

BFS 在网格上求的就是“最少步数”。

方向数组：网格题的通用套路

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// 四方向：下、上、右、左
const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};

// 遍历四个邻居
for (int k = 0; k < 4; ++k) {
    int nx = x + dx[k];
    int ny = y + dy[k];
    // 依次检查越界、障碍、是否已访问
}
```

八方向只需把数组扩成 8 个元素，其余代码不变。

代码：网格最短路


```cpp
const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};

// g[x][y] == '#' 表示障碍；返回最少步数，不可达返回 -1
int bfsGrid(vector<string>& g, int sx, int sy, int tx, int ty) {
    int n = g.size(), m = g[0].size();
    vector<vector<int>> d(n, vector<int>(m, -1));
    queue<pair<int,int>> q;

    d[sx][sy] = 0;
    q.push({sx, sy});

    while (!q.empty()) {
        auto cur = q.front();
        q.pop();
        int x = cur.first, y = cur.second;

        if (x == tx && y == ty) return d[x][y];
```

代码：网格最短路


```
19
20          for (int k = 0; k < 4; ++k) {
21              int nx = x + dx[k], ny = y + dy[k];
22              if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
23              if (g[nx][ny] == '#' || d[nx][ny] != -1) continue;
24              d[nx][ny] = d[x][y] + 1;
25              q.push({nx, ny});
26          }
27      }
28      return -1;
29  }
```

目录

1 BFS 基本思想

2 BFS 的实现

3 网格上的 BFS

4 经典例题

5 总结



例题一：无权图最短路

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

问题描述

- 给定无权图，求 s 到 t 的最少边数
- 边权全部为 1

做法

- 从 s 出发 BFS
- 输出 $dist[t]$

要点

- 每个点第一次被访问时
- 距离就是最短距离
- 一旦 t 被访问，就可以停止

例题二：迷宫最少步数

问题描述

- $n \times m$ 网格，# 是墙，. 是路
- 从起点走到终点，每步上下左右走一格
- 求最少步数

建模范式

- 格子是点，相邻是边
- 用方向数组现算邻居
- `d[x][y]` 记录到 (x, y) 的最短距离

要点：越界、障碍、访问三连判

例题三：数连通块

问题描述

- 给定无向图，求连通块个数
- 每个块内的点两两可达

做法

- 遍历所有点
- 遇到未访问点就 BFS 一整块
- 计数器加一

要点

- BFS 把整块标记为“已访问”
- 遍历结束时计数器就是块数
- 与 DFS 数连通块效果一致

总复杂度仍为 $O(V+E)$

例题四：按层处理

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

问题描述

- 有些题需要“一层一层”地处理
- 例如：腐烂的橘子、最小传播时间

关键技巧

- 每轮开始先记录当前队列长度 sz
- 只处理 sz 个点
- 这 sz 个点就是同一层

层数 + 1，就是传播所需的时间

目录

1 BFS 基本思想

2 BFS 的实现

3 网格上的 BFS

4 经典例题

5 总结

BFS 核心要点

要点速记

- 队列驱动，先进先出
- 入队即标记，绝不重复
- 逐层扩展，距离递增
- 边权相等时，必得最短路

复杂度

- 时间 $O(V+E)$
- 空间 $O(V)$
- 每个点入队一次
- 每条边被检查常数次

BFS = 队列 + 访问标记 + 逐层扩展。

BFS 与 DFS 的选择

优先选 BFS

- 求无权最短路
- 需要按层处理
- 状态空间分层明显

场景	DFS	BFS
无权最短路	否	**是**
连通性	是	是
拓扑排序	是	否
层序信息	否	**是**
内存敏感	**是**	否

易错点速查

易错点	后果与修复
出队时才标记	同一点多次入队，复杂度退化 $\rightarrow$ 入队即标记
忘记初始化	残留旧值 $\rightarrow$ 每轮重置 `dist`
无向图单边加边	邻居不全 $\rightarrow$ 两个方向都要加
多层循环忘了分层	层次信息丢失 $\rightarrow$ 先记队列长度
网格越界未判	数组越界 $\rightarrow$ 先判边界再访问

附录：为什么 BFS 是 $O(V+E)$

点的角度

- 每个点最多入队一次
- 最多出队一次
- 出队操作总计 $O(V)$

边的角度

- 点 u 出队时遍历 `adj[u]`
- 有向图：每条边被看一次
- 无向图：每条边被看两次

总计 $O(V)+O(E)=O(V+E)$。

附录：一份完整可运行代码


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);          // 无向图加两条
    }

    vector<int> dist(n + 1, -1);
    queue<int> q;
    dist[s] = 0;
    q.push(s);                        // 入队即标记
```

附录：一份完整可运行代码


```
19
20      while (!q.empty()) {
21          int u = q.front(); q.pop();
22          for (int v : adj[u]) {
23              if (dist[v] != -1) continue;
24              dist[v] = dist[u] + 1;
25              q.push(v);
26          }
27      }
28
29      cout << dist[t] << "\n";            // 不可达时输出 -1
30      return 0;
31  }
```